



República Bolivariana de Venezuela
Ministerios del Poder Popular para la Educación Universitaria
Universidad Central de Venezuela
Facultad de Ciencias
Escuela de Computación



Asalto Rápido

Proyecto 2 Algoritmo y Estructuras de Datos

Auxiliares Docentes:

Bryan Silva

Erimar Reis

Estudiante:

Elsa Pestana

Caracas, 25 de febrero de 2026

INFORME

El Equipo Delta reporta que ha culminado con éxito la actualización del sistema armamentístico con nuevas mejoras en topología, buscando la manera de preservar la mayor energía de los autómatas de batalla X78A-Cazador. Para ello, se ha logrado implementar un motor físico que evalúa el desgaste energético de las unidades durante su desplazamiento, garantizando de esta manera que los exploradores alcancen su destino con la mayor cantidad de batería posible, evitando rutas mortales o caminos inconclusos. De esta forma, se asegura la conservación estratégica de los recursos y el éxito absoluto en las misiones por venir.

1-Análisis del Problema

El primer aspecto técnico abordado fue la naturaleza del entorno de exploración. El terreno donde operan los autómatas no es plano, sino un espacio tridimensional. Esto implica que las bases y campamentos a los que se dirigen actúan como puntos estratégicos en un mapa, poseyendo una identificación única y coordenadas espaciales exactas en los ejes X, Y y Z. Para llegar a dichos lugares, existe una red de caminos que los conectan, los cuales cuentan con un coeficiente de "fricción" específico que representa los diferentes obstáculos y condiciones del suelo que el autómata pueda conseguir en su trayecto.

Al momento de iniciar sus misiones, los autómatas cuentan con una cantidad de energía finita, la cual variará durante todo el trayecto y resultará crucial para llevar a cabo el despliegue con éxito. Esta variación energética está condicionada por dos factores físicos principales. En primer lugar, el coeficiente de fricción presente en todos los caminos, el cual exige siempre un gasto constante de batería durante el movimiento. Por otro lado, la gravedad juega un rol fundamental en el cálculo, ya que las subidas y bajadas a través de los puntos estratégicos implican un desgaste o ganancia importante de energía. Finalmente, los autómatas operan bajo una regla de supervivencia estricta: si en algún punto del trayecto la energía cae por debajo de cero, el autómata se detiene, descartando esa ruta y declarando la misión como un fracaso.

Por otro lado, gracias a las mejoras implementadas, los autómatas no recorren los caminos de forma aleatoria, sino que establecen analíticamente cuál es la mejor ruta para llevar a cabo su objetivo. La ruta ideal siempre será aquella que le permita llegar a su destino con la mayor energía remanente. Sin embargo, pueden presentarse ocasiones donde el autómata encuentre dos o más caminos que resulten en la misma cantidad de energía final. Para resolver esto, el autómata activa un protocolo de desempate: primero, evalúa cuál es la ruta que requiere recorrer una menor distancia (que se entiende como la menor cantidad de puntos estratégicos visitados); si persiste la igualdad, verifica y selecciona el camino que presente la menor cantidad de cambios de altura, garantizando la ruta más estable. Adicionalmente, esta mejora estructural dota al autómata de la capacidad para detectar si se encuentra geográficamente aislado, identificando al instante si no existen caminos viables para llegar a su objetivo.

2- Diseño

2.1 Precisión numérica y manejo de flotantes

Uno de los principales problemas técnicos fue realizar las comparativas para efectuar los desempates, dado que tanto la energía, como la distancia y el cambio de altura son variables de tipo flotante (**double**). Esto implicó un problema grave, puesto que las comparaciones directas de igualdad o desigualdad entre flotantes resultaban inexactas

debido al estándar IEEE 754; a medida que el número acumula operaciones, pierde precisión al momento de representarlo.

Para solucionar este problema se optó por implementar un margen de tolerancia numérica o **Epsilon**. De este modo, se evalúan dos casos:

1. Cuando la variable es estrictamente menor:

```
if(energia < arbol->energia - epsilon){
```

2. Cuando la variable es estrictamente mayor.

```
if(energia > arbol->energia + epsilon)
```

Si el valor no encaja en alguno de estos dos casos, el algoritmo deduce matemáticamente que ambas cantidades son equivalentes y baja al nivel siguiente de desempate.

Complementando con lo anteriormente mencionado, el concepto de tolerancia se aplicó para corregir los límites de supervivencia. Al agotar la energía de forma exacta, el sistema generaba residuos negativos microscópicos por el cálculo binario, rechazando la ruta. Al evaluar la energía sobrante permite un margen mínimo (**energia >= -epsilon**), el cual logró que el autómata reconociera correctamente el **0.00** absoluto como una llegada válida.

2.2 Cola de Prioridad y desempates

Para que el autómata pudiera decidir cuál era el camino más óptimo durante su explicación, se implementó un registro dinámico de las rutas evaluadas y la energía restante en cada una. Para gestionar este registro, se desarrolló la clase **ColaPrioridad**. Es importante destacar que a pesar de llamarse Cola de Prioridad, no es una Cola de Prioridad Tradicional, sino un árbol binario de búsqueda. Esta estructura fue diseñada con una lógica de clasificación específica: los candidatos más eficientes (aquellos con mayor energía o que ganan los desempates) se sitúan en las ramas derechas mientras que las rutas menos viables se relegan a la izquierda.

2.2.1 Estructura del Nodo y gestión de memoria

Para que los autómatas mantengan un registro exhaustivo de las rutas exploradas, se diseñó la estructura **NodoArbol**. Esta no se concibió únicamente como un componente de un árbol binario, sino como una unidad de almacenamiento que encapsula el estado físico del autómata en un punto específico del mapa. La toma de decisiones del algoritmo se fundamenta en las siguientes variables almacenadas en cada nodo:

- **id**: Identifica el nodo del grafo al cual corresponde la posición actual.
- **energía**: Registra la carga remanente tras descontar el trabajo de fricción y el cambio de energía gravitacional.
- **distancia**: Almacena la longitud acumulada del recorrido desde el punto de origen.
- **cambioAltura**: Cuantifica la sumatoria absoluta de los cambios de elevación vertical.

```
struct NodoArbol{
    int id; // id del nodo
    double energia; // contad
    double distancia; // dist
    double cambioAltura; // c
    NodoArbol* nodoDerecho; /
    NodoArbol* nodoIzquierdo;
};
```

Para la instanciación de estos elementos, se implementó la función **crearNodo**. Su cometido principal es recibir las métricas físicas calculadas durante la exploración y empaquetarlas mediante la asignación de memoria dinámica. Un aspecto crítico de esta función es el manejo de los punteros **NodoDerecho** y **NodoIzquierdo**, los cuales preparan el nodo para su posterior integración en la estructura del árbol. Este proceso garantiza que cada trayectoria sea tratada como un objeto independiente, asegurando la integridad de los datos frente a la naturaleza recursiva del algoritmo de búsqueda.

```
NodoArbol* crearNodoArbol(int id, double energia, double distancia, double cambioAltura){
```

2.2.2 Insertar y desempates

El alma de la clase ColaPrioridad es la función insertar, dado que en ella se ejecutan los diferentes niveles de desempate y se toma la decisión de cuál nodo va a la derecha y cuál va a la izquierda.

```
void insertar(NodoArbol *&arbol, int id, double energia, double distancia, double cambioAltura){
```

Lo primero que se hace dentro de la función es preguntar si **arbol** es nulo, ya que si lo es llama a la función **crearNodo** para crear un nuevo nodo.

```
if(arbol == NULL){
    arbol = crearNodoArbol(id, energia, distancia, cambioAltura);
    return;
}
```

Luego se comienzan aplicar los 3 niveles de desempate

Nivel 1-Prioridad de energía: El autómata sabe que el camino ideal siempre será aquel que lo deje con mayor energía, por lo que si la energía del nuevo nodo es estrictamente mayor al del nodo actual del árbol, lo posiciona a la derecha y si por el contrario es menor lo sitúa a la izquierda.

```
if(energia < arbol->energia - epsilon){
```

```
if(energia > arbol->energia + epsilon)
```

Nivel 2-Primer desempate (Distancia): Cuando la diferencia de energía del nodo actual con el nodo nuevo es igual se aplica el primer desempate, en este caso sitúa el nodo que lo deja con menor distancia recorrida a la derecha y el que lo deja con mayor distancia lo sitúa a la izquierda.

```
if(distancia < arbol->distancia - epsilon){
```

```
if(distancia > arbol->distancia + epsilon)
```

Nivel 3-Segundo desempate (Altura): Si las energías son iguales y las distancias también, se aplica el último método de desempate el cual consiste en colocar al nodo que produzca menos cambios de altura a la derecha y el que lo deja con mayores cambios de altura lo sitúa a la izquierda.

```
if(cambioAltura < arbol->cambioAltura - epsilon)
```

De esta forma, se crea un árbol binario donde los mejores nodos siempre irán a la derecha y los peores a la izquierda.

2.2.3 Obtener mejor Nodo

Una vez que el árbol ha organizado todas las rutas posibles, el algoritmo de Dijkstra requiere extraer siempre el "mejor" candidato para continuar la exploración. Gracias a la lógica de inserción explicada anteriormente, no es necesario recorrer todo el árbol; el camino con mayor prioridad se encuentra sistemáticamente en el nodo situado más a la derecha de la estructura.

La función **sacarMejor** opera bajo un principio de búsqueda extrema: se desplaza de forma iterativa a través de los punteros derechos hasta alcanzar un nodo que ya no posea hijos a su derecha.

```
NodoArbol* sacarMejor(NodoArbol *&arbol)
```

Este método de extracción garantiza que el autómata siempre trabaje con la ruta más prometedora en cada paso, reduciendo drásticamente el número de iteraciones necesarias para encontrar el destino final.

2.3 Algoritmo de Dijkstra

El núcleo del sistema de navegación de los autómatas consiste en una adaptación del algoritmo de Dijkstra. Su propósito es encontrar la ruta más óptima entre dos puntos estratégicos del mapa, priorizando la supervivencia energética del autómata. Este algoritmo opera evaluando cada paso de la ruta más prometedora disponible en la **ColaPrioridad**.

2.3.1 Estructuras de Soporte

Para que el algoritmo gestione la memoria de la exploración de forma eficiente, se implementaron dos estructuras de datos dinámicas, que actúan como "libretas" donde el autómata va anotando su recorrido:

- **libreta1Energias**: Actúa como un registro de la máxima energía con la que se ha logrado alcanzar cada nodo. Solo se permite actualizar un nodo si la nueva ruta ofrece una energía mayor a la registrada previamente
- **libreta2Caminos**: En esta libreta se almacena el ID del punto estratégico predecesor, lo que permite reconstruir la secuencia lógica de pasos una vez que el algoritmo alcanza el punto estratégico de destino.

2.3.2 Explorador

El proceso de exploración del autómatas se realiza en un ciclo continuo que solo se detiene al agotar las opciones en el árbol o al encontrar el destino.

```
void exploracion(ColaPrioridad& explorador, double libreta1Energias[], int libreta2Camino[], int destino)
{
    while(explorador.raiz != nullptr){
```

Dentro del ciclo se realizan 4 etapas fundamentales para obtener la ruta más óptima. Las cuales son:

1. Se le solicita a la clase ColaPrioridad sacar el mejor punto estratégico registrado hasta el momento

```
NodoArbol* arbolActual = explorador.sacarMejor(explorador.raiz);
```

2. Posteriormente se almacenan en variables temporales los datos de dicho punto estratégico (**id, energia, distancia, cambioAltura**)

```
int id = arbolActual->id;
double energia = arbolActual->energia;
double distancia = arbolActual->distancia;
double camAltura = arbolActual->cambioAltura;
```

3. Luego se recorre el mapa desde el punto estratégico actual para identificar todos los caminos posibles hacia puntos estratégicos adyacentes, después de conseguir los puntos estratégicos adyacentes se calculan las diferentes funciones físicas para determinar cuánta energía consumirá el autómatas en ese tramo.

```
double disEntreNodos = distancia3D(nodos[id], nodos[idVecino]);

double egGravitacional = EGravitacional(energia, distancia, camAltura);
double wFriccion = Wfriccion(caminoActual, caminoVecino);
double energiaActual = EFinal(energia, egGravitacional, wFriccion);
```

4. Antes de tomar un camino como óptimo se verifica si la energía resultante es mayor o igual a epsilon y si a su vez es mayor a la mejor energía ya registrada en la libreta de energías. Si se cumplen todas estas condiciones el camino es insertado al árbol y se actualizan las libretas

```
if(energiaActual >= -epsilon && energiaActual > libreta1Energias[idVecino]){

    if(fabs(energiaActual) < 1e-6) {
        energiaActual = 0.0;
    }

    libreta1Energias[idVecino] = energiaActual;
    libreta2Camino[idVecino] = id;

    double nuevaDistancia = distancia + 1;

    double camAlturaNew = camAltura;

    if(nodos[id].z != nodos[idVecino].z){
        camAlturaNew++;
    }

    explorador.insertar(explorador.raiz, idVecino, energiaActual, nuevaDistancia, camAlturaNew);
}
```

Conclusión

A lo largo de este informe se detalló paso a paso de cómo la integración de cálculos físicos, estructuras como árboles binarios, permitieron desarrollar un sistema de navegación capaz de priorizar la supervivencia energética de los autómatas X78A-Cazador, logrando así alcanzar sus objetivos con la mayor cantidad de batería posible. El uso de Epsilon para gestionar la impresión de los puntos flotantes fue determinante para la implementación correcta de los niveles de desempate y validación de rutas críticas. De esta forma se garantiza la obtención de rutas óptimas para llevar a cabo los objetivos de los autómatas, mediante una adaptación del algoritmo de Dijkstra y el uso de la estructura del árbol binario nombrado anteriormente para la gestión de prioridades, el sistema selecciona analíticamente la ruta ideal basándose en la energía remanente, la distancia recorrida y los cambios de elevación.

Los autómatas X78A-Cazador con la mejora en navegación reposan en el almacén de la base MST de umaril, donde alguna vez estuvieron listos para pelear por la recuperación de umaril.